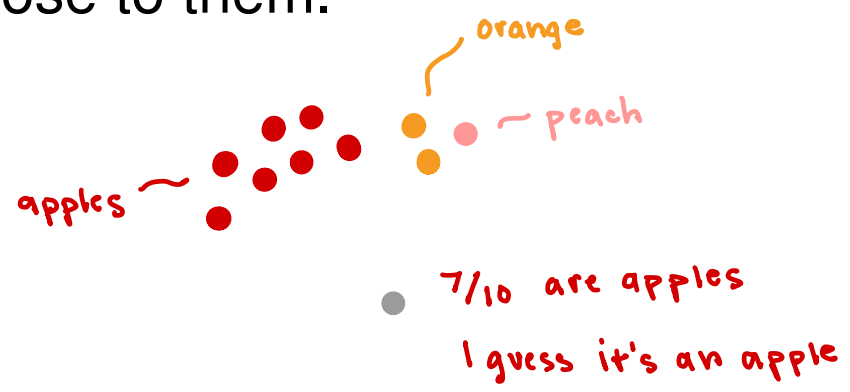
Week 6, k-Nearest-Neighbors

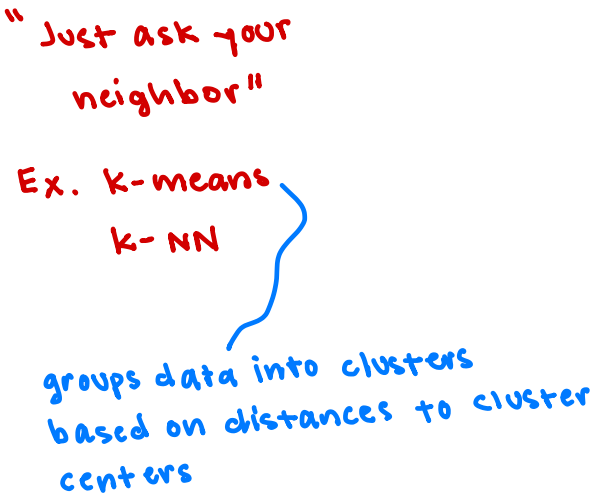
CAPP 30254, Machine Learning for Public Policy

Distance-based classification

A *distance-based* machine learning model is a model that makes classifications by using distances between data points.

The basic assumption in distance-based learning is that similar things tend to be near each other. So, to classify a new example, a model has points look around themselves at nearby examples and guesses a label for the new example based on what's close to them.





k-Nearest-Neighbors

k-N-N
KNN

k-Nearest-Neighbors (k-NN) is a distance-based classification model that classifies a data point $\mathbf{x}$ by finding the k closest neighbors of $\mathbf{x}$ and predicts $\mathbf{x}$'s label based on the labels of its k closest neighbors.

k-Nearest-Neighbors

k-Nearest-Neighbors (k-NN) is a distance-based classification model that classifies a data point $\mathbf{x}$ by finding the k closest neighbors of $\mathbf{x}$ and predicts $\mathbf{x}$'s label based on the labels of its k closest neighbors.

k-NN starts with a labeled training set $(\mathbf{x}_i, y_i)$. To predict the label y of a new data point $\mathbf{x}$:

- Find the distance between $\mathbf{x}$ and every training point $\mathbf{x}_i$
- Select the k nearest training points to $\mathbf{x}$
- Pick the most frequent label out of the k training points
- Assign that label to the new data $\mathbf{x}$

k-NN is a lazy learner

little/no work at training
lots of work at prediction
- k-NN: distance between $\underline{x}$ and every $\underline{x}_i$

k-NN is a *lazy learner*. Lazy learners wait until they're asked to make a prediction to do any work.

Other algorithms we've seen are called *eager learners*. Eager learners build a model during training.

For example, ridge regression computes the model **w** during training. Then it can make fast predictions because the hard work was already done.

RR: training: minimize $\ell(\underline{w})$ to find the optimal $\underline{w}$

$$\ell(\underline{w}) = \sum_i (y_i - \underline{w}^T \underline{x}_i)^2 + \lambda \|\underline{w}\|_2^2$$

gradient descent — hyperparameter

prediction: $\underline{w}^T \underline{x}$ done!
= y the prediction

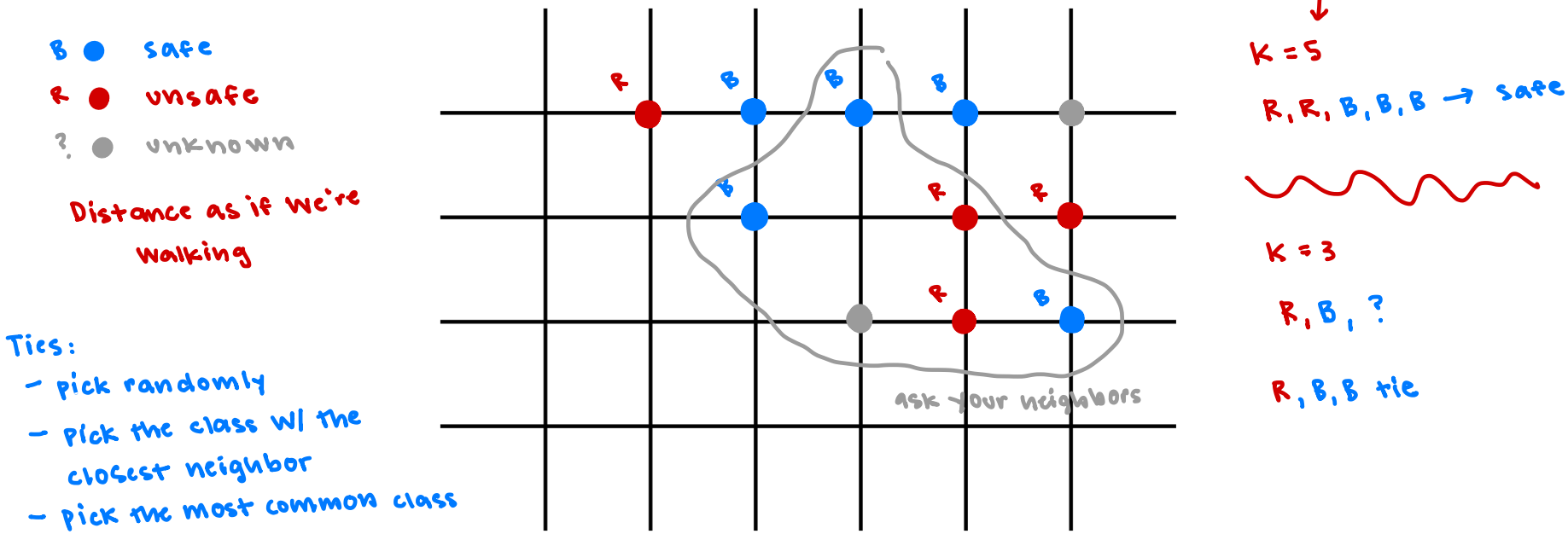
Example

Is an intersection is safe or unsafe for pedestrians?

lots of features

Recall that features like lighting and walk signals determine whether or not an intersection is safe for pedestrians.

Say that we know the safety classification of most of the intersections in Springfield. Let's determine the safety of an unclassified intersection using k-NN.



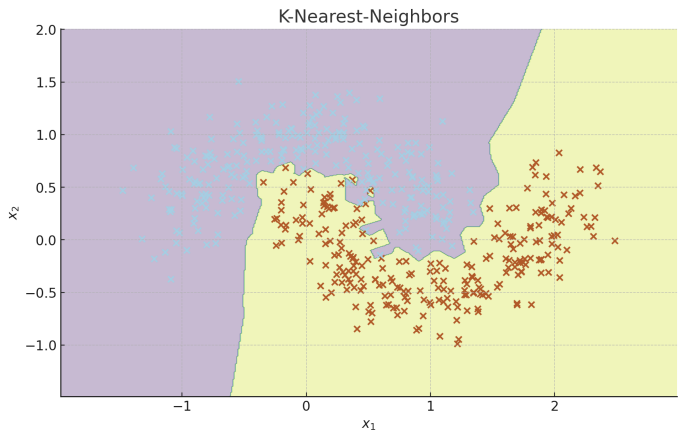
Choosing the right k

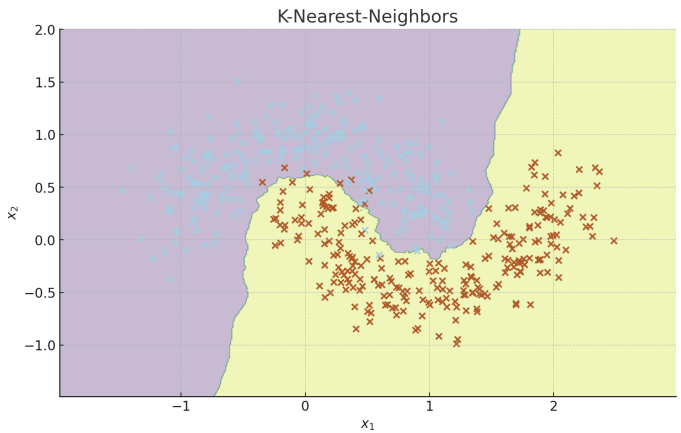
noise affects predictions
- model fits noise instead of true pattern
- for k=1 - only trusts closest neighbor

Choosing the right k in k-NN is important since it directly controls how the model behaves.

- A small k may capture noise and lead to overfitting
- A large k may underfit the data

averaging over more neighbors
- lose local patterns / not capture details
- blur classes together





Which plot do you think was made with a small value for k?

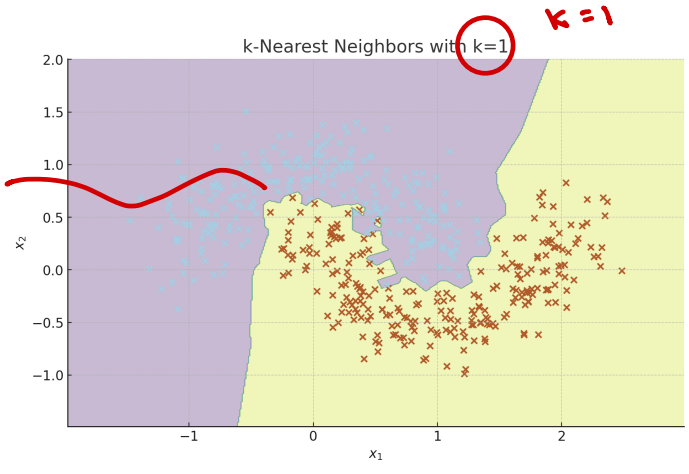
Choosing the right k

k=1 tries to perfectly fit the training data

k=10 generalizes more, less sensitive to noise

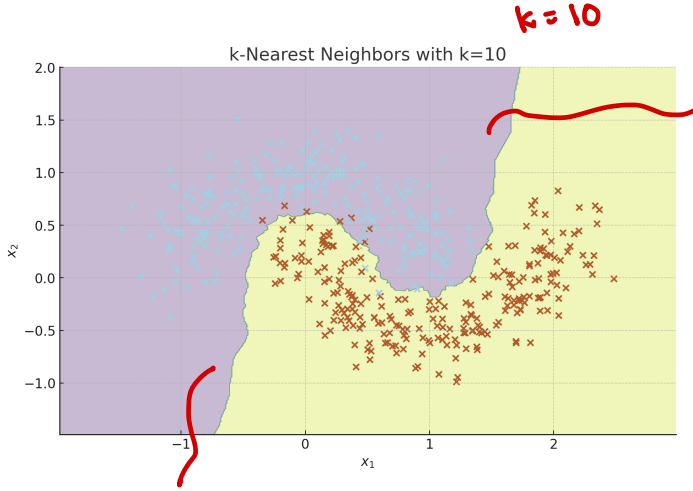
Choosing the right k in k-NN is important since it directly controls how the model behaves.

- A small k may capture noise and lead to overfitting
- A large k may underfit the data



k=1

jagged



k=10

smooth boundary

nonlinear

[1]

[1] Generated by ChatGPT with the prompts: “Can you draw a plot of k nearest neighbors with k=1?”, “... k=10?”

How do you choose the right k?

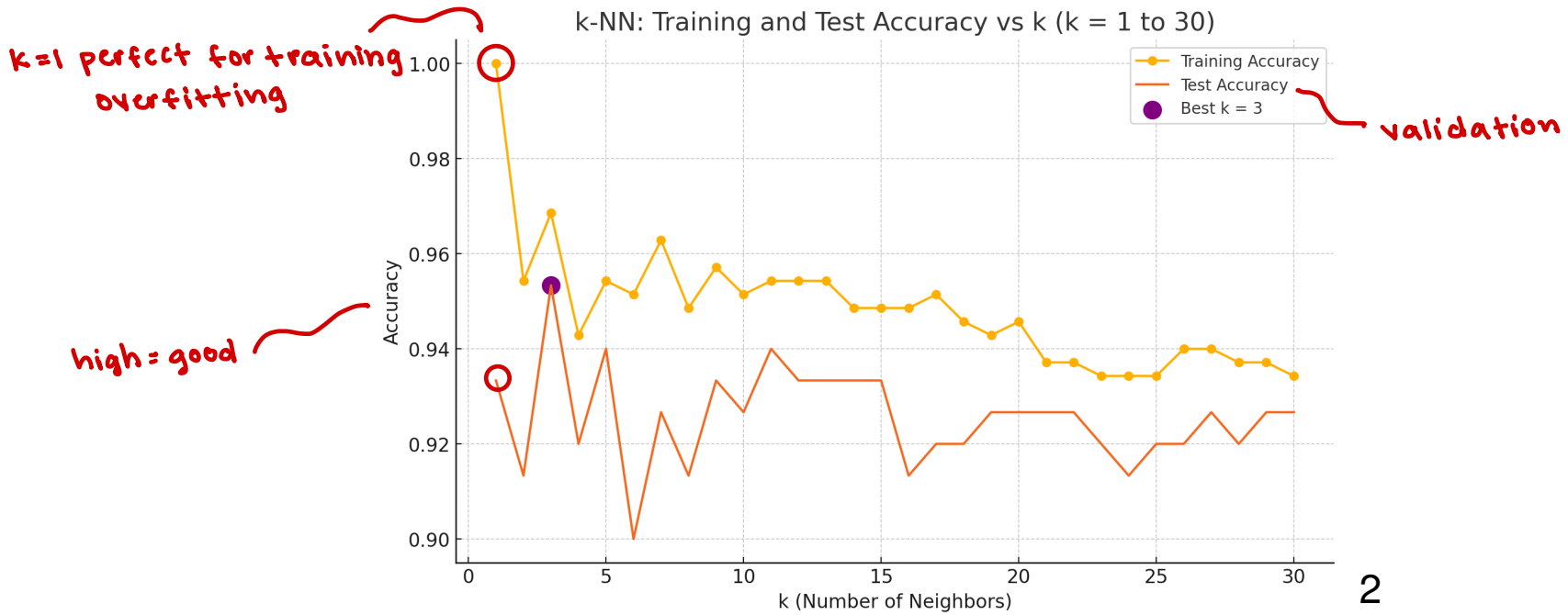
How should you choose the value of k?

[2] Generated by ChatGPT with the prompt: “Can you plot the accuracy of knn k=1 to 30?”

How do you choose the right k?

How should you choose the value of k? Cross-validation!

Here's a plot of the accuracy of k-NN model with $k = 1, \ldots, 30$.



[2]

[2] Generated by ChatGPT with the prompt: "Can you plot the accuracy of knn k=1 to 30?"

Distance metrics

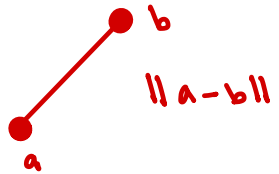
$\|a-b\| = \sqrt{(a_1-b_1)^2 + \ldots + (a_n+b_n)^2}$

$d(a,b) = |a_1-b_1| + \ldots + |a_n-b_n|$

For distance-based models, we need to choose a metric to measure the distance between two items.

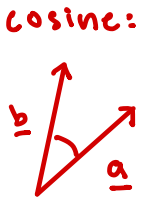
Some common distance metrics are:

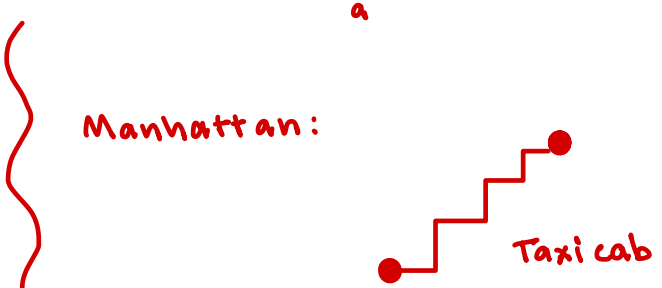
- Euclidean distance
- Manhattan distance
- Cosine similarity



standard

for grids





Important! Always normalize features when you're using k-NN. Large features can misleadingly dominate computations.

$\cos(\theta) = \frac{\underline{a} \cdot \underline{b}}{\|\underline{a}\| \cdot \|\underline{b}\|}$

almost always

Pros and cons of k-NN

Pros:

- k-NN is simple to implement and easy to understand
- Works easily with multi-class classification
- It can model non-linear decision boundaries

Cons:

- Computationally expensive and requires a lot of storage
- Can be biased if the training set is imbalanced
- Suffers from the "curse of dimensionality"

Curse of dimensionality

k-NN suffers from the so-called "curse of dimensionality". The curse of dimensionality is a curse that occurrs when you work with very high dimensional data.

↳ data w/ lots of features

In high dimensions, points spread out so much that the distance between them is not very meaningful. Notions like "close" and "neighbor" don't really mean anything anymore.

like planets

This is a problem for distance-based models like *k*-NN and clustering models because they depend on neighborhoods.

Fix: more data or feature selection / PCA

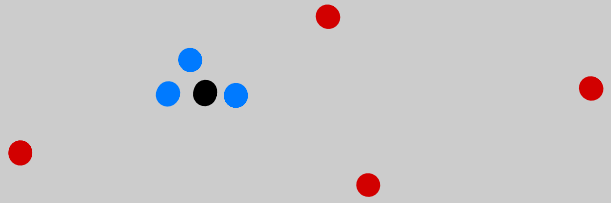
Imagine 100 points randomly in:

- 1D $[0,1]$ randomly scattered
  1 dim. Plenty of points! Points have close neighbors
- 2D $[0,1] \times [0,1]$ (the square box)
  2 dim. points are spread thinner. Neighbors are farther away
- 10D
  10 dim. Now 100 points in a 10D cube. Points are incredibly far apart
  ↳ AKA 10 features space is almost entirely empty

Why it matters for KNN:

- K-NN needs nearby neighbors to work well
- In high dim. everything is far apart
- NO "closest points" unless you have an enormous amount of data

Weighted *k*-NN



k = 7 should the unknown really be classified as red?

In regular *k*-NN, each neighbor of a new data point **x** contributes the same amount (1) to the classification of **x**.

In *weighted k-NN*, neighbors that are closer to **x** make a larger contribution to the classification of **x** than neighbors farther away.

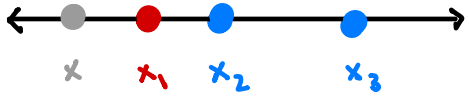
The most common way contribution weight is computed is by using the inverse distance:

$$weight = \frac{1}{distance}$$

far away: less influence, small weight

close: more influence, weight = big

data	class	dist.	weight
x_1	R	0.5	2.0
x_2	B	1.0	1.0
x_3	B	2.0	0.5



Regular: x = B

Weighted: x = R!

k-NN vs. k-means

	k-Nearest-Neighbors	**k-Means**
Type	Supervised learning (classification)	Unsupervised clustering
Purpose	Predict a label for new data points	Group data into clusters
How it works	Find the k closest labeled examples and pick most occurring label	Assign points to one of k clusters – based on the distance to the cluster centers
Training	No training phase (lazy learner)	Learns cluster centers
Input	Labeled data (features + labels)	only features – no labels
Output	Predicted label for new data	cluster assignment – which cluster new data belongs to

[3]

[3] Generated by GPT-4, DALL-E 3 after the prompt: “Hi, could you please make a minimalistic logo for a machine learning class...”